SHREEJI REPOSITORY

Technical & Server Architecture Analysis

End-to-end review of the website, API server, request/response handling, database persistence, file storage, cloud configuration, security and deployment readiness.

Repository	**github.com/founder-svg/Shreeji**
Branch reviewed	main
Review date	2 November 2026
Review type	Static source-code and configuration review

BOTTOM LINE

A structured Express API exists, but the reviewed repository is not yet proven production-ready as an integrated cloud application. Important blockers include exposed administrator fallback credentials, unsafe KYC data handling, incomplete SQL migrations, front-end API-host inconsistencies, and a deployment workflow that does not deploy.

Prepared for technical review and sharing. No repository files were changed.

1. Executive summary

The repository contains a real-estate/property-consultancy website and a Node.js/Express server. The backend also retains an older e-commerce API and SQL schema. The most important issue is not that Express is absent: it is that the website, API routes, persistence layer, deployment configuration and security controls do not consistently form one working production system. [S1-S4]

Area	Assessment	What this means
Server structure	Reasonable foundation	Express middleware, versioned routes, controllers, repositories, health endpoints and JWT middleware are present.
Website-to-API connection	Not reliable	Different frontend pages choose different API base URLs; one public inquiry path targets localhost in visitors' browsers.
PostgreSQL readiness	Incomplete	The committed initial migration is commerce-focused while many current real-estate repositories expect additional tables.
Cloud storage	Optional capability only	S3-compatible storage is implemented, but the checked-in Compose configuration sets local storage.
Security & privacy	Critical concerns	Hardcoded admin fallbacks and plaintext Aadhaar/PAN handling require immediate remediation.
Deployment verification	Not demonstrated	The deploy workflow only prints a message; no successful live backend health check was established.

Overall verdict

Do not treat the current repository as a verified production deployment. The Express server is a meaningful starting point, but security, schema completeness, API host configuration and actual deployment must be fixed and verified before relying on it for customer inquiries or identity documents.

Highest-priority findings

- Critical - Administrator authentication: source code contains fixed administrator login fallbacks that can mint admin JWTs. Remove the fallback path, rotate any exposed credentials and invalidate existing sessions. [S8]
- Critical - KYC privacy: the KYC controller creates a masked Aadhaar string but passes the complete 12-digit value to the repository; Aadhaar and PAN are persisted in ordinary columns. No application-level encryption is visible in these code paths. [S9, S30]
- Critical - Database schema drift: property/client/KYC/project/plot/media/audit repositories expect tables absent from the committed initial migration. The order repository also writes `idempotency_key`, which that migration's `orders` table does not define. [S7, S10]
- High - Frontend API destinations: production pages do not consistently point to a real backend origin; one form calls the visitor's `localhost`, while another chooses a GitHub Pages URL as API host. [S12, S13, S14]
- High - Deployment: GitHub Actions deploy workflow is a notification/echo job; the actual SSH deployment steps are commented out. [S19]

What is working in principle

The backend has a conventional request pipeline, API versioning, CORS allow-list, JSON parsing, request IDs, timeouts, request logging, rate limiting, JWT authentication, password hashing for client accounts, PostgreSQL connection pooling, migration execution, and S3-compatible upload support. Those are useful building blocks; their presence does not, by itself, prove a correctly configured live environment. [S3-S6, S15]

2. Scope, translation and review method

Gujarati instruction - English translation: “Give the repository link to an AI for analysis. Also check how the server responds, what the incoming and outgoing responses are, how data is sent from the server to the website and from the website to the server, how it is handled in the cloud, and in what language or format the data is stored.”

This report follows that scope. It traces both directions of communication, examines route registration and controller behavior, checks repository/database code and file storage, and reviews deployment/test configuration.

Method and limits

- Reviewed the public GitHub `main` branch source files, README, package manifests, SQL migration, Docker configuration, GitHub Actions workflows, tests and the website/portal/admin/3D-map code.
- Followed key flows from browser `fetch()` calls through Express route registration, controllers and repository methods to PostgreSQL/local JSON/file storage, then back through JSON responses.
- Classified implementation evidence separately from configuration or runtime assumptions. Source references are listed at the end of this report.
- Not performed: no source changes, deployment, database login, cloud-bucket login, real customer submission, admin login against a live instance, or full test-suite execution. The report is therefore a static code/configuration review, not a penetration test or a measured live-server test.

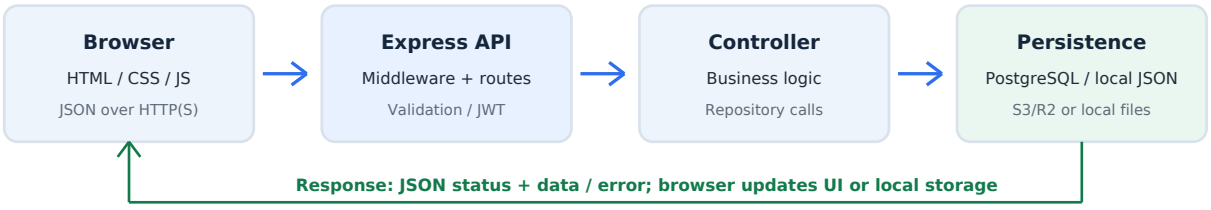
> **Important runtime limitation**
>
> The repository does not provide a verified production API hostname and the committed deployment workflow does not execute a deployment. Therefore this review can explain how the server is coded to respond, but cannot honestly certify that a public server is currently online, healthy, connected to a remote database, or accepting live traffic.

A mismatch in project identity

The root README describes Shreeji as a luxury e-commerce/retail platform, and the server manifest and initial SQL migration retain that e-commerce vocabulary. The visible website title and workflows instead describe commercial real-estate consulting, plots, property inquiries, client accounts and KYC. The backend has grown real-estate endpoints, but the docs/schema have not been brought into full alignment. This is a significant maintenance and deployment risk. [S1, S2, S7, S20]

3. Architecture and technology stack

IMPLEMENTED CONCEPTUAL DATA PATH (CODE REVIEW)



Important: the diagram describes server-side code paths; actual production host, database and bucket are not verified.

Main components

Layer	Technology / implementation	Observed responsibility
Website	HTML, CSS, browser JavaScript	Public real-estate pages, forms, client portal and admin interface. The 3D map loads local Three.js libraries and embedded JS geometry/data.
3D viewer	Three.js, OrbitControls, GLTFLoader, GSAP	Renders local model/texture/project data in the browser; the page includes local project data script files rather than a clear API-fed model bootstrap.
Backend runtime	Node.js (ES modules), Express 4	Listens on configurable port (default 5000), mounts /api/v1 and legacy /api routes.
Middleware	Helmet, CORS, compression, Morgan, JSON/body parser, request ID, timeout, rate limiter	Security headers, cross-origin handling, request parsing, tracing/logging and traffic controls.
Authentication	JWT + bcryptjs	Admin and client tokens; protected routes expect Authorization: Bearer <token>. Client registration hashes passwords with bcrypt.
Relational database	PostgreSQL via pg; compatible with Supabase Postgres	Parameterized SQL, connection pool and SQL migrations when DATABASE_URL is present.
Fallback persistence	In-process JavaScript objects + JSON file	Local store is persisted to server/data/local-db.json when PostgreSQL is not active.
File storage	S3-compatible API / local filesystem	AWS S3, Cloudflare R2 or Supabase S3-compatible endpoint if configured; otherwise local server/uploads.

API shape and versioning

The app mounts the same router at both /api/v1 and /api, so the preferred versioned base is /api/v1. The root route returns service metadata (service name, status, version and endpoint names). Health endpoints are separate from business APIs: liveness checks the process; readiness pings the database and reports storage provider metadata. [S3, S4]

Server request pipeline: step by step

This is the full inbound and outbound path implemented by the Express application. Each stage below shows where data is read, checked, transformed or persisted.

- 1. Browser request: website sends an HTTP request, usually JSON with `Content-Type: application/json`; protected routes include a Bearer JWT.
- 2. Express middleware: CORS allow-list, Helmet, compression, body parsing (10 MB limit), request ID, timeout, timing and request logging run before route logic.
- 3. Route and guards: versioned route maps the URL/method to a controller and applies rate limits, validation and/or authentication/role checks.
- 4. Controller: reads `req.body`, `req.params`, `req.query` and authenticated user claims; validates input and invokes a repository or service.
- 5. Repository/service: runs parameterized SQL against PostgreSQL when active, otherwise accesses `localStore` and may write a JSON file. Uploads pass through the storage service.
- 6. Response: controllers serialize JSON with a status code. Global not-found/error middleware standardizes failures; browser code parses JSON and updates the UI/local storage.

4. Server request and response behaviour

The backend is designed around REST-style HTTP endpoints. JSON is the main interchange format for normal API calls. Data is not “stored in JavaScript” merely because JavaScript runs the server: JavaScript is the implementation language, JSON is the API payload format, and SQL/PostgreSQL types are used for database persistence. [S2-S7]

Use case	Method + path	Protection	Expected response pattern
Liveness	GET /api/v1/health/live	Public	200 JSON with status, uptime and timestamp.
Readiness	GET /api/v1/health/ready	Public	200 when DB ping succeeds, otherwise 503; includes database/provider/latency and storage provider fields.
Property lead	POST /api/v1/property-inquiries	Public + submission limiter	201 JSON with success/message, lead ID/name/phone/status/createdAt and optional WhatsApp dispatch metadata.
Client registration	POST /api/v1/client/register	Public + auth limiter	201 JSON with success, signed JWT and a safe client profile; password hash is not returned.
Client KYC	POST /api/v1/client/kyc	Client Bearer JWT + limiter	JSON request with identity and property details; response contains an application status/object.
Admin media metadata	POST /api/v1/admin/media	Admin Bearer JWT	JSON metadata record; this is not the binary file upload endpoint.
Actual file upload	POST /api/v1/uploads	Admin Bearer JWT + upload limiter	Multipart upload handled by Multer, then stored to local disk or configured S3-compatible storage.
Products	GET /api/v1/products	Public + API limiter/cache	200 JSON: success, data array and pagination metadata.
Order creation	POST /api/v1/orders	Public + validation/submission limiter	201 JSON order on success; PostgreSQL path uses transaction, DB-side product price and stock reservation.

Example HTTP exchange: request and response

The following is an illustrative payload based on the controller contract, not a captured request to a live production server.

Browser-to-server JSON request

```
{
  "name": "Example Customer",
  "phone": "+919876543210",
  "email": "customer@example.com",
  "role": "buyer",
  "propertyCategory": "Commercial",
  "preferredLocation": "Gandhidham",
  "builtupArea": "2500 sq ft",
  "message": "Looking for a warehouse",
  "source": "shreeji_web_portal"
}
```

Server-to-browser JSON response (illustrative)

```
{
  "success": true,
  "message": "Your commercial inquiry has been registered successfully...",
  "data": {
    "id": "lead-...",
    "name": "Example Customer",
    "phone": "+919876543210",
    "status": "new",
    "createdAt": "<ISO timestamp>"
  },
  "whatsAppDispatch": {
    "dispatchId": "...",
    "phoneNumber": "...",
    "directUrl": "https://wa.me/..."
  }
}
```

Authentication and errors

Client registration uses bcrypt hashing and returns a signed JWT. Protected requests are expected to send Authorization: Bearer <token>. The server has a central error handler that returns JSON such as { "success": false, "error": "..." } and includes stack details only in development mode. However, the admin login controller’s hardcoded fallback path undermines the intended admin password verification and must be removed. [S3, S4, S8, S26]

5. Where data is stored, and in what format

The repository supports several storage modes. The important distinction is between a capability implemented in code and a cloud resource that is actually configured. The checked-in environment defaults to local storage; no real cloud account, database endpoint, bucket name with valid credentials, or live storage response was independently verified. [S5, S6, S15, S24]

Data	Storage path in code	Format / representation	Assessment
Products, categories, orders	PostgreSQL repository path	Relational SQL columns; numeric, varchar/text, boolean, timestamps; JSONB for images/tags/specs and shipping address.	Sound overall pattern, but orders.idempotency_key is missing from the committed orders table.
Property inquiries, client accounts, KYC, projects, plots, map assets, media metadata	PostgreSQL repositories when DB active	Relational rows with IDs, text, status, timestamps and foreign-key-like identifiers.	Repositories expect tables not defined in the committed initial migration; deployment needs complete migrations.
Local server data	localStore + persistLocalStoreAsync()	JavaScript objects serialized as UTF-8 JSON into server/data/local-db.json.	Useful for development/tests; not a durable shared production database across multiple instances.
Browser-side drafts/session	Website localStorage and sessionStorage	String values, frequently JSON.stringify()/JSON.parse() records; tokens stored as strings.	Per-browser only; does not sync to the server automatically. JWT in localStorage has XSS exposure considerations.
Images, video, PDF, CAD and other files	Storage service	Binary objects; metadata/URLs held as JSON or database fields.	S3-compatible path supported, but default configuration is local filesystem /uploads.
Background jobs	In-process queue + pending-jobs.json	Job objects serialized as JSON on local disk.	Not a distributed queue; some handlers only log success rather than deliver notifications.
Cache	In-process LRU	Runtime memory entries	Per-process cache; not shared between server replicas.

PostgreSQL and SQL/JSONB

When DATABASE_URL is configured, the server uses the Node pg driver and a connection pool, then scans the migrations directory and applies unapplied .sql files. PostgreSQL is the actual database engine; SQL is used to define/query tables. Some semi-structured fields use JSONB, which stores JSON data in PostgreSQL. This is a sensible hybrid model for structured customer/order data and flexible product metadata. [S5-S7]

Local fallback

When no database URL is set in non-production mode, the server loads a JavaScript object store and saves it to local-db.json. That file is JSON text, not PostgreSQL. Several repositories (including cart) use the local store directly. This does not provide a common database across multiple instances and can create split-brain or lost-update behaviour if the application is scaled horizontally. [S6, S11]

Cloud file storage and deployment reality

The storage service can instantiate an S3 client for AWS S3, Cloudflare R2 or a Supabase S3-compatible endpoint when a non-local provider and access/secret keys are configured. If not, it writes binary files to the server’s local uploads directory and returns a /uploads/... URL. The checked-in Compose service explicitly sets STORAGE_PROVIDER=local; presigned direct uploads are unavailable without S3/R2 configuration. [S15, S24]

Answer to “which language is the data stored in?”

Server logic is JavaScript (Node.js). API bodies and local persistence files are JSON. The relational database stores records using PostgreSQL SQL data types, with JSONB columns for flexible nested data. Uploaded photos, videos, PDFs and CAD assets are stored as binary files/objects. Browser-only state is stored as strings in localStorage/sessionStorage.

6. Website-to-server integration: actual findings

The real-estate API endpoints do exist in the full route table. The key problem is inconsistent routing from different front-end areas, plus fallbacks that make a local/demo action look like a successful server operation. [S4, S12-S14]

Frontend area	Browser behaviour	Finding
Public buyer/seller inquiry helper	POSTs JSON to /property-inquiries; on non-local host the API base is https://founder-svg.github.io/Shreeji/api/v1.	Route exists, but GitHub Pages is a static-site origin rather than an Express backend origin unless a separate proxy is configured. No such proxy/backend mapping is evidenced in the repo.
Alternative inquiry submit handler	Saves inquiry in browser localStorage, then POSTs to http://localhost:5000/api/v1/inquiries.	On a visitor's production device, localhost points to that visitor's own computer, not the hosted Shreeji server. The request also omits email and message, which the generic inquiry controller calls .trim() on. UI does not verify response.ok before showing the success state.
Client portal	Uses relative /api/v1 for registration/login/me/KYC and saves token + client profile in localStorage.	Works only if the static site host routes /api/v1 to the Express server. No backend proxy is included in the repository deployment workflow. Offline registration fallback creates a fake, unsigned token and marks the client locally authenticated.
Admin panel	Uses localhost in local development and same-origin /api/v1 in production; saves some content locally and attempts JSON API sync.	Same-origin API needs a reverse proxy or shared host; not configured in committed deploy workflow. The fallback admin experience can be local/demo-only rather than a server-authenticated session.
Media studio	Creates URL.createObjectURL(file) for a selected upload and sends metadata JSON to /admin/media.	The selected binary file is not sent to /uploads; the blob URL is browser-session scoped and cannot serve as a permanent URL for other users.
3D map	Loads Three.js, geometry/model/texture/project JS data files directly from local paths.	The inspected page bootstraps from embedded static data, not a clearly wired live projects/plots API call. Admin database edits may not automatically appear in this viewer.

Why a form can show success even when the server did not save it

The buyer/seller helper saves a local browser copy and invokes the property-inquiry helper asynchronously. A second handler uses a different API URL and only logs the HTTP status; it does not check response.ok before proceeding. Network errors are caught, but a non-2xx response is not treated as a failed submission in the same way. This is a user-trust issue: the website may report success while the cloud database did not receive a lead. [S12, S13]

Recommended single integration pattern

- Define one environment-based `API_BASE_URL` for all pages (for example a dedicated `https://api.` origin), and never use GitHub Pages or browser localhost as the production API host.
- Create one shared API helper that sends JSON, includes authentication when required, checks `response.ok`, parses structured errors and uses a consistent request ID.
- Remove offline "success" fallbacks from real customer workflows. Store a local draft only if it is clearly labelled pending sync and retry safely when the actual server confirms receipt.
- Upload the actual `File` as `multipart/form-data` to `/api/v1/uploads` (or request a presigned S3/R2 URL), then save the returned stable asset URL as metadata.

7. Critical and high-severity findings

CRITICAL Administrator login bypass

The backend contains fixed administrator identifiers/passwords as fallback accounts and signs a full admin JWT when they match. This bypasses the normal persisted password-hash check. A demo/autofill credential is also present in admin UI code. Remove all fallback credentials, rotate exposed secrets, invalidate sessions, and audit admin actions. [S8, S14]

CRITICAL Aadhaar and PAN stored without visible application encryption

The KYC controller validates full Aadhaar, constructs a masked value for presentation, but passes the complete Aadhaar to the repository; the repository writes the full value into `aadhar_number`, and PAN into `pan_number`. Treat as sensitive identity data. Stop collecting it until necessary protections exist; encrypt at field level with managed key rotation, minimize access, restrict logs, define retention/deletion and perform a privacy review. [S9, S30]

CRITICAL PostgreSQL migration does not match current real-estate repositories

The inspected initial migration defines commerce tables (categories, products, admin users, orders, order items and inquiry data), but current repositories query tables such as `property_inquiries`, `client_users`, `client_kyc_applications`, `projects`, `plots`, `map_assets`, `project_media` and `audit_logs`. On a fresh database initialized only from this migration, real-estate calls will fail unless additional schema is provisioned outside the repo. [S7, S18, S31]

HIGH Production API host is inconsistent

One public helper uses the GitHub Pages URL as API base; a second inquiry submitter points to browser localhost. Portal/admin use relative same-origin paths. These three patterns will not consistently reach one deployed Express instance. Configure a single real backend host and proxy all pages through it. [S12-S14]

High-severity findings (continued)

The following items are production blockers even where a feature appears to work in a local/demo environment.

HIGH Production Docker Compose does not inject required secrets

The Compose file sets `NODE_ENV=production` but lists only port/CORS/cache/storage variables. It merely comments that `DATABASE_URL`, `JWT_SECRET` and `ADMIN_PASSWORD` should be set via `.env`; it does not use `env_file` or pass those variables. The env validator makes all three mandatory and exits when absent. A clean checkout will therefore fail unless operators inject them through an external override/environment. [S5, S24]

HIGH Order write expects a missing column

The PostgreSQL order repository selects and inserts `idempotency_key`, but the committed initial `orders` table has no such column. The transactional order insert will fail against that schema. Add a migration and unique/appropriate index, or remove the repository dependency. [S7, S10]

HIGH Media studio does not upload selected files

The browser creates a `blob:` object URL and stores it in localStorage, then POSTs the URL and metadata as JSON. That URL is local to the browser context and is not a durable public object. Use the existing multipart upload route or presigned cloud upload and store the stable returned URL. [S14, S15]

HIGH Notifications are not actually delivered

The WhatsApp service constructs `wa.me` / WhatsApp Web links and stores a dispatch record; background job handlers for email, lead and WhatsApp print to console and return success-like values. This is not proof of automatic message delivery. Integrate a provider API/webhook, add delivery receipts/retries, and report queued vs delivered states distinctly. [S16, S17]

HIGH Audit log persistence is incomplete

The audit repository inserts into `audit_logs`; the initial migration does not define that table. Insert errors are caught and logged, so auditing can silently fail from the caller’s perspective; the admin list query may still throw. Add migration and test write/read behaviour. [S7, S18]

8. Additional operational and quality findings

Topic	Code evidence	Operational effect / action
Cart persistence	Cart repository uses only localStore.cart and persists through local JSON; it has no PostgreSQL branch.	Cart state is tied to one server process/disk volume. It is not a shared relational cart store for multiple replicas. Persist cart to Postgres with session/user ownership. [S11]
Fake offline portal account	Registration catch creates fallback client state after API failure and stores a demo token in browser localStorage.	The fallback token is not a JWT signed by the server. Any protected KYC request using it should fail authentication if it reaches the API. Mark demo mode explicitly; never say the production account was created. [S26]
3D viewer data is static	Inspected 3D page imports local model, texture and per-project JavaScript data scripts.	This gives fast static rendering but not a live database-fed digital twin by itself. Add API fetch + loading/error states + versioned model assets if real-time admin changes are expected. [S21]
Process-local queue/cache	Job queue persists pending tasks to local JSON and the LRU cache exists in the server process.	Multiple PM2/container instances will not share queue/cache state; a durable Redis-backed queue and shared cache are preferable for scaled production. [S17, S22]
PM2 mode	PM2 config uses exec_mode: fork and one instance.	README language about cluster mode/throughput is not reflected in this configuration. Use an intentional process strategy only after fixing state sharing and validating DB limits. [S22]
Tests	npm test runs four scripts; the Postgres test and load test use pg-mem, an in-memory PostgreSQL emulator. Other integration tests use seeded/local fixtures.	Useful unit/integration coverage, but it does not prove compatibility with a real hosted PostgreSQL instance, S3/R2 bucket, production secrets, TLS, domain routing or external notification APIs. [S2, S23]
Deployment workflow	Workflow checks out code and echoes an “initiating deployment” message. SSH commands are commented out.	A push to main does not deploy the server through this workflow. Add the real deployment path or document the external deployment process and health-check it. [S19]

What can be said about server response quality?

- Positive: route handlers generally return JSON; property inquiry returns HTTP 201 on accepted submissions; health/readiness endpoints distinguish process liveness from database connectivity; API and auth rate limiters are wired; common not-found/error middleware is registered. [S3, S4]
- Not established: the repository does not prove that the currently intended frontend URL routes to the actual Express process, that PostgreSQL schema has all required tables, that uploads reach a cloud bucket, or that notifications are delivered to a human. Those require integration/runtime verification.
- Performance: README claims are not treated as live performance measurements. The test harness uses an in-memory database emulator and localhost process; its results cannot establish real cloud latency, throughput, database contention or uptime. [S23]

Server “working or not?” conclusion

The Express implementation is structurally present and can plausibly serve requests when correctly configured. However, based on the committed files reviewed, the complete public website-to-server-to-database-to-cloud journey is not reliable or demonstrated. Several production paths will fail, remain local-only, or falsely report success. Live operation remains unverified.

9. Prioritized remediation plan

Priority	Action	Acceptance check
P0 - Immediate	Delete backend admin credential fallbacks and admin demo bypasses. Rotate all exposed credentials/secrets; expire admin JWTs; inspect access logs and audit for unexpected admin use.	No static credential path in source. Admin login only succeeds against a stored bcrypt hash. Negative tests reject all former fallback values.
P0 - Immediate	Stop storing raw Aadhaar/PAN. Decide whether this collection is needed; minimize fields and retention. Apply vetted field-level encryption/tokenization and restrict admin access.	Database contains no unencrypted full Aadhaar/PAN. KYC access is role-scoped; tests prove sensitive fields are never returned to general client endpoints or logs.
P0 - Schema	Create complete versioned migrations for all real-estate entities and operational tables. Add idempotency_key and indexes to orders. Make migrations deterministic and tested on a blank Postgres database.	A fresh real PostgreSQL database can start from zero migrations and pass CRUD tests for leads, clients, KYC, projects, plots, map assets, media, audits and orders.
P0 - Networking	Choose one production backend hostname. Centralize API base config for website, portal and admin; remove production localhost/GitHub Pages API targets; add HTTPS reverse proxy/CORS origin.	Every frontend flow reaches the same deployed /api/v1 server. Browser network traces show real host, expected method/path, 2xx response and persisted row.
P1 - Forms	Repair generic inquiry payload/validation contract. Ensure required name/email/message fields are present or controller handles optional values. Only show submitted success after verified 2xx and durable server confirmation.	Test buyer, seller and landlord forms from a non-local browser; all create exactly one persisted lead; failed API calls show pending/error, not false success.
P1 - File upload	Connect media studio to /uploads multipart or S3/R2 presigned upload. Store durable URL only after upload completes; remove blob URLs from persistent metadata.	Upload from admin, close browser, open asset from another device and confirm it renders. Verify access control, file type/size limits and delete behaviour.
P1 - Cloud runtime	Set production DATABASE_URL, strong JWT secret, rotated admin credential, explicit STORAGE_PROVIDER, S3/R2 endpoint/keys, allowed origins, and Compose env_file/environment mapping. Prefer managed Postgres and object storage.	Deployment boots from clean clone with documented secret injection. Readiness check reports expected provider and DB round trip. File upload returns durable cloud URL.
P1 - Messaging	Use real email/WhatsApp provider integration with delivery state, retry, timeout and observability. Keep “link generated”, “queued”, “sent” and “delivered” as separate states.	A sandbox recipient receives one real test message; provider event/webhook recorded; failed jobs retry and are visible to admins.
P2 - Reliability	Move queue/cache to shared infrastructure (e.g. Redis), use a durable job processor, and define whether multiple server instances are supported.	Restart/scale test proves jobs are not lost or duplicated; cache invalidation works across instances.
P2 - Verification	Run automated tests against real PostgreSQL and object storage in CI/staging. Implement a real deploy job and post-deploy health/readiness smoke test.	CI proves migrations and core API flows against real Postgres; deploy workflow performs a deployment and fails when post-deploy health check fails.

Minimum go-live checklist

Before the application is trusted with live customers or identity documents, each of these checks should pass in a production-like staging environment.

- No hardcoded administrator passwords, secrets or demo bypasses in tracked source.
- Aadhaar/PAN protected with reviewed collection, encryption, access and retention controls.
- Fresh PostgreSQL database fully migrates and all current repository tables exist.
- One canonical HTTPS API origin used by every site page; no production browser localhost calls.
- Real media upload path tested from a separate device; no `blob:` URL used as stored content.
- Production secrets actually injected into container; storage provider and database provider confirmed by readiness check.
- Real email/WhatsApp delivery tested or UI clearly states manual handoff only.
- CI and deployment perform real infrastructure tests and post-deploy health checks.

10. Final assessment and source register

Final assessment

Shreeji has a recognisable backend architecture: Express routes, middleware, controller/repository separation, JWT protection, PostgreSQL support and an S3-compatible upload adapter. Those are real implementation elements, not merely a diagram. The strongest positive part is the server-side separation of API routes, controller logic and persistence code, plus the attempt to provide health checks, input rate limits and database transactions for orders. [S3, S4, S10, S15]

The blocking issue is the gap between what the code supports and what the project is actually configured to run. The visible website is a property-consultancy experience, while the primary migration/docs still reflect e-commerce. The production frontend API addresses are inconsistent; real-estate tables are not represented in the committed initial schema; the cloud storage default is local; and the deploy workflow is not a deployment. Security and KYC handling need immediate attention before customer or identity data is processed.

> **Overall recommendation**
>
> Treat the current version as a development prototype / partially integrated system. Do not rely on it for real KYC collection or production lead/order processing until the P0 items are fixed and the go-live checks above pass against an actual hosted environment.

Source register

The following links point to the public repository files reviewed. Line anchors are included where practical. The source files are the basis for the code-level findings; line references may shift if the main branch changes after this review.

[S1] README and product identity - README.md
[S2] Server package, dependencies and test scripts - server/package.json#L1-L58
[S3] Express app, middleware, API mount and health endpoints - server/src/app.js#L16-L125
[S4] Full API route table, including real-estate/client/KYC/media routes - server/src/routes/api.js#L42-L228
[S5] Environment validation, DB/JWT/admin/storage settings - server/src/config/env.js#L7-L83
[S6] Local JSON store, Postgres initialization and migration runner - server/src/config/database.js#L20-L215
[S7] Committed initial SQL migration - server/migrations/001_initial_schema.sql#L1-L107
[S8] Admin auth controller and fallback administrator accounts - server/src/controllers/authController.js#L18-L82
[S9] KYC controller and client repository persistence - server/src/controllers/clientKycController.js#L42-L72
[S10] Order repository insert and idempotency key - server/src/repositories/orderRepository.js#L116-L247
[S11] Cart repository uses localStore only - server/src/repositories/cartRepository.js#L0-L39
[S12] Main website API base and property inquiry helper - Shreeji web/script.js#L238-L268
[S13] Alternative inquiry handler and client portal fallback/auth state - Shreeji web/script.js#L1148-L1209
[S14] Admin panel API base and media studio blob URL flow - Shreeji web/admin/index.html#L1165-L1527
[S15] S3/R2/Supabase-compatible file storage implementation - server/src/config/storage.js#L12-L120
[S16] WhatsApp link generation and dispatch record - server/src/services/whatsappService.js#L84-L135
[S17] Background job handlers log notification actions rather than send them - server/src/jobs/jobQueue.js#L19-L40
[S18] Audit repository SQL and persistence handling - server/src/repositories/auditRepository.js#L26-L66
[S19] GitHub Actions deploy workflow (deployment commands commented out) - .github/workflows/deploy.yml#L1-L33
[S20] Website page title and real-estate product identity - Shreeji web/index.html#L1-L10
[S21] 3D map page loading local Three.js and embedded project data - Shreeji web/3d-map/index.html#L13-L24
[S22] PM2 configuration: fork mode, one instance - server/ecosystem.config.cjs#L12-L35
[S23] Postgres and load tests use pg-mem; performance harness uses seeded/local fixtures - server/tests/postgres.test.js#L1-L70
[S24] Docker Compose production variables and local storage setting - docker-compose.yml
[S25] Dockerfile build/runtime defaults - server/Dockerfile#L1-L29
[S26] Portal API/fallback and client registration auth - portal.html / clientAuthController.js
[S27] Property inquiry controller validation and 201 response - server/src/controllers/propertyInquiryController.js#L4-L72
[S28] Generic inquiry controller expects name, email and message - server/src/controllers/inquiryController.js#L28-L60
[S29] CI workflow installs dependencies and runs npm test - .github/workflows/ci.yml#L1-L34
[S30] Client repository writes the full Aadhaar/PAN fields to database columns - server/src/repositories/clientRepository.js#L77-L145
[S31] Real-estate repositories query the current domain tables (property leads, projects, plots, map assets and media) - server/src/repositories/propertyInquiryRepository.js#L65-L116